EXPERIMENT-7

Temporal Difference Learning using TD(0) Algorithm

Name: AKHILA POCHIMI REDDY

Reg no: 192425225

Course code: MLA0305

Course Name: REINFORCEMENT LEARNING

Slot: B

Aim

To implement the **TD(0) Temporal Difference Learning algorithm** and estimate the value of states through repeated interaction with the environment.

Procedure

1. Create a **Grid World** with START and GOAL states.
2. Initialize the state-value function.
3. Select actions and observe the next state and reward.
4. Update the state value using the **TD(0) update rule**.
5. Repeat the process for multiple episodes.
6. Generate the learned **START → GOAL path** and visualize the learning performance.

Python code:

```
import numpy as np

import pandas as pd

import matplotlib.pyplot as plt

from matplotlib.patches import Rectangle

from IPython.display import display

# ============================================================

# EXPERIMENT 7
```

```
# Temporal Difference Learning using TD(0)
# ========================================================
np.random.seed(42)
N = 4
START = 0
GOAL = 15
alpha = 0.1
gamma = 0.9
episodes = 300
actions = {
   0: (-1, 0),   # Up
   1: (1, 0),    # Down
   2: (0, -1),   # Left
   3: (0, 1)     # Right
}
arrows = {
   0: "↑",
   1: "↓",
   2: "←",
   3: "→"
}
# ========================================================
# STATE FUNCTIONS
# ========================================================
```

```
def position(s):
    return divmod(s, N)
def get_state(row, col):
    return row * N + col
# ============================================================
# ENVIRONMENT
# ============================================================
def move(s, action):
    if s == GOAL:
        return GOAL
    row, col = position(s)
    dr, dc = actions[action]
    nr = max(0, min(N - 1, row + dr))
    nc = max(0, min(N - 1, col + dc))
    return get_state(nr, nc)
def reward(s):
    return 10 if s == GOAL else -1
# ============================================================
# TD(0) LEARNING
# ============================================================
V = np.zeros(N * N)
learning_history = []
for episode in range(episodes):
    state = START
```

```
    total_reward = 0
    for step in range(50):
        # Random action
        action = np.random.randint(4)
        next_state = move(state, action)
        r = reward(next_state)
        # TD(0) update
        td_target = r + gamma * V[next_state]
        td_error = td_target - V[state]
        V[state] += alpha * td_error
        total_reward += r
        state = next_state
        if state == GOAL:
            break
    learning_history.append(total_reward)
# ============================================================
# LEARNED POLICY
# ============================================================
policy = np.zeros(N * N, dtype=int)
for s in range(N * N):
    if s == GOAL:
        continue
    values = []
    for action in range(4):
```

```
        ns = move(s, action)
        values.append(
            reward(ns) + gamma * V[ns]
        )
    policy[s] = np.argmax(values)
# ============================================================
# FIND START → GOAL PATH
# ============================================================
path = [START]
state = START
for _ in range(30):
    if state == GOAL:
        break
    state = move(state, policy[state])
    path.append(state)
# ============================================================
# SUMMARY TABLE
# ============================================================
summary = pd.DataFrame({
    "Parameter": [
        "Episodes",
        "Learning Rate (α)",
        "Discount Factor (γ)",
        "Path Length",
```

```
        "Start State Value"
    ],
    "Value": [
        episodes,
        alpha,
        gamma,
        len(path) - 1,
        round(V[START], 3)
    ]
})
print("\n" + "=" * 50)
print("            TD(0) SUMMARY")
print("=" * 50)
display(summary)
# ============================================================
# STATE VALUE TABLE
# ============================================================
value_table = pd.DataFrame({
    "State": [f"S{i+1}" for i in range(N * N)],
    "TD(0) Value": np.round(V, 3)
})
print("\n" + "=" * 50)
print("           STATE VALUES")
print("=" * 50)
```

```
display(value_table)

# ============================================================

# START → GOAL VISUALIZATION

# ============================================================

fig, ax = plt.subplots(figsize=(7, 7))

ax.set_xlim(-0.5, N - 0.5)

ax.set_ylim(N - 0.5, -0.5)

ax.set_aspect("equal")

for row in range(N):

    for col in range(N):

        rect = Rectangle(

            (col - 0.5, row - 0.5),

            1,

            1,

            fill=False,

            linewidth=2

        )

        ax.add_patch(rect)

        s = get_state(row, col)

        if s == START:

            ax.text(

                col, row - 0.2,

                "START",

                ha="center",
```

```
            fontsize=10,
            fontweight="bold"
        )
        ax.text(
            col, row + 0.18,
            "●",
            ha="center",
            fontsize=18
        )
    elif s == GOAL:
        ax.text(
            col, row,
            "GOAL",
            ha="center",
            va="center",
            fontsize=11,
            fontweight="bold"
        )
    else:
        ax.text(
            col, row,
            arrows[policy[s]],
            ha="center",
            va="center",
```

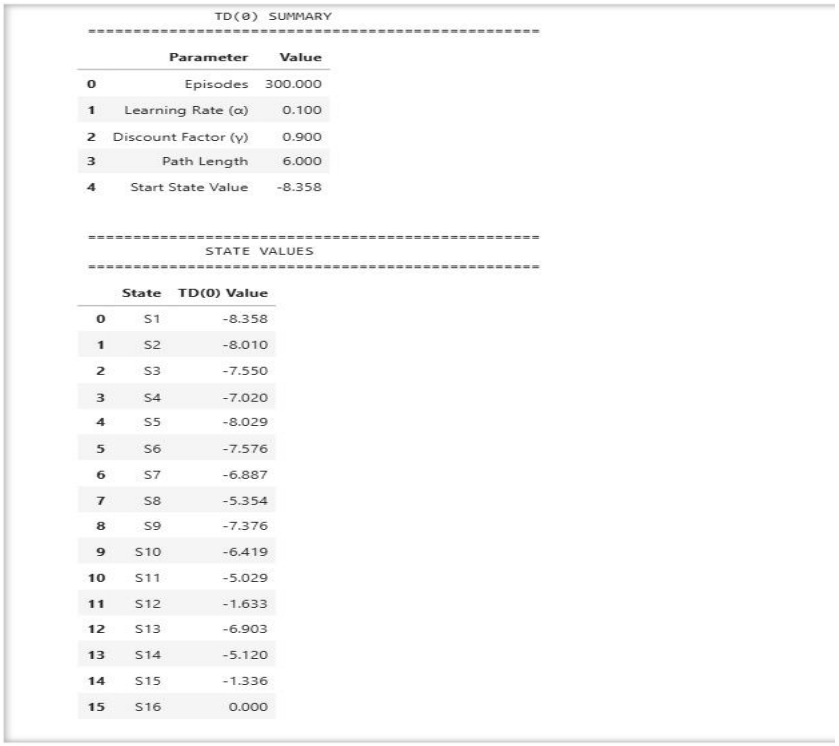
```
            fontsize=24,
            fontweight="bold"
        )
# Draw learned path
px = [position(s)[1] for s in path]
py = [position(s)[0] for s in path]
ax.plot(
    px,
    py,
    linestyle="--",
    linewidth=2,
    marker="o",
    markersize=5
)
ax.set_title(
    "TD(0) Learning: START → GOAL",
    fontsize=15,
    fontweight="bold"
)
ax.set_xticks([])
ax.set_yticks([])
plt.tight_layout()
plt.show()
# ============================================================
```

```
# LEARNING PERFORMANCE GRAPH
# ============================================================
window = 20
moving_average = np.convolve(
    learning_history,
    np.ones(window) / window,
    mode="valid"
)
plt.figure(figsize=(9, 5))
plt.plot(
    range(1, len(moving_average) + 1),
    moving_average,
    linewidth=2
)
plt.xlabel("Episode")
plt.ylabel("Average Reward")
plt.title("TD(0) Learning Performance")
plt.grid(
    True,
    linestyle="--",
    alpha=0.4
)
plt.tight_layout()
plt.show()
```

```
# ============================================================

# RESULT

# ============================================================

print("\n" + "=" * 50)

print("                RESULT")

print("=" * 50)

print(

   "TD(0) learning was successfully implemented. "

   "The state values were updated using the TD error, "

   "and an improved START → GOAL policy was obtained."

)
```

Output:

Summary



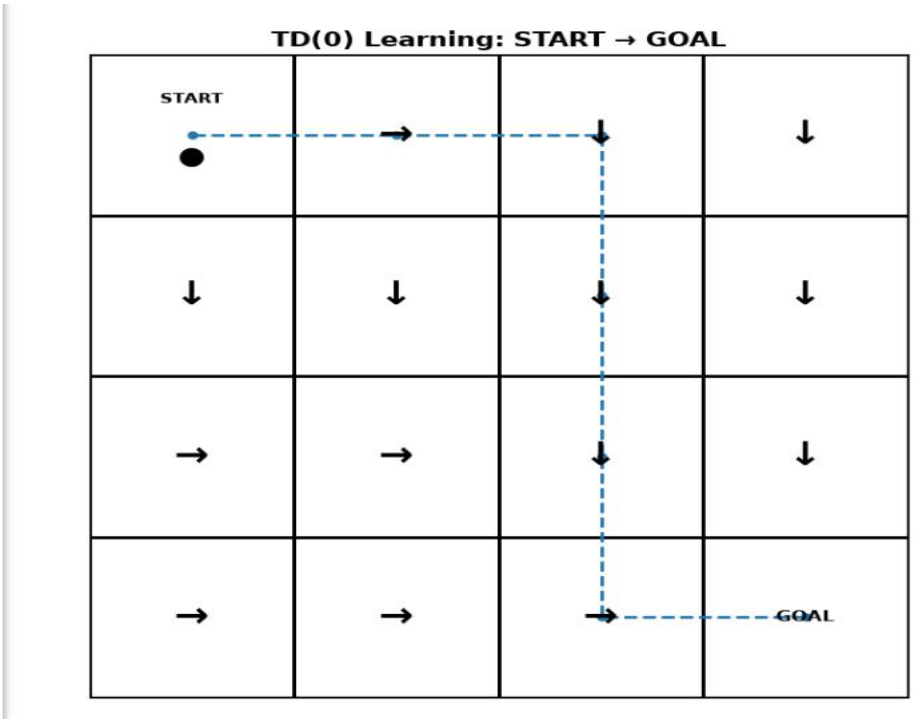
TD(0) SUMMARY

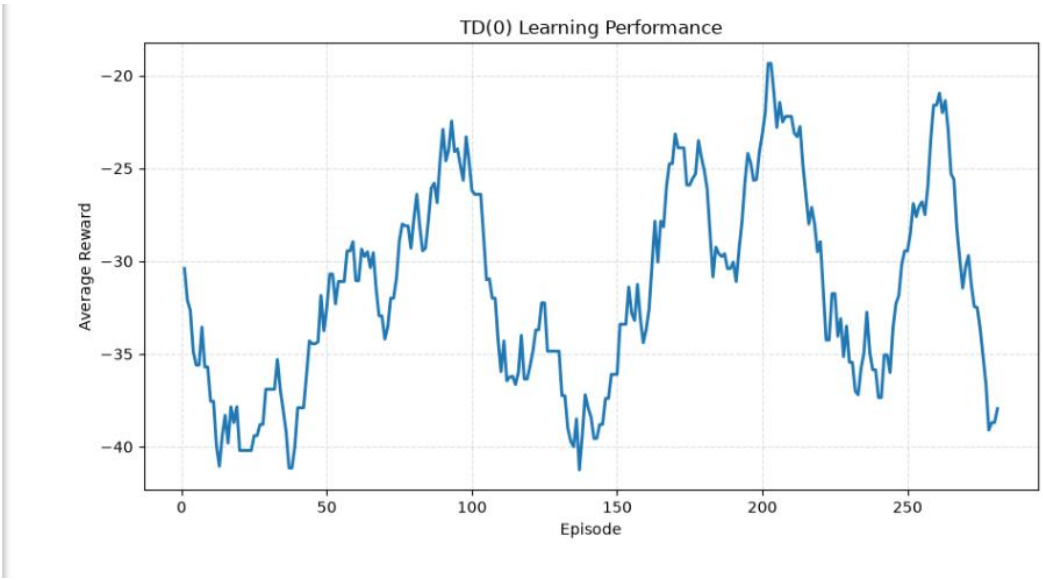
	Parameter	Value
0	Episodes	300.000
1	Learning Rate (α)	0.100
2	Discount Factor (γ)	0.900
3	Path Length	6.000
4	Start State Value	-8.358

STATE VALUES

	State	TD(0) Value
0	S1	-8.358
1	S2	-8.010
2	S3	-7.550
3	S4	-7.020
4	S5	-8.029
5	S6	-7.576
6	S7	-6.887
7	S8	-5.354
8	S9	-7.376
9	S10	-6.419
10	S11	-5.029
11	S12	-1.633
12	S13	-6.903
13	S14	-5.120
14	S15	-1.336
15	S16	0.000

Visualization:





Result

The **TD(0) algorithm was successfully implemented**. The state values were updated through repeated episodes, and the learned policy successfully produced a **START → GOAL path** with its learning performance visualized.